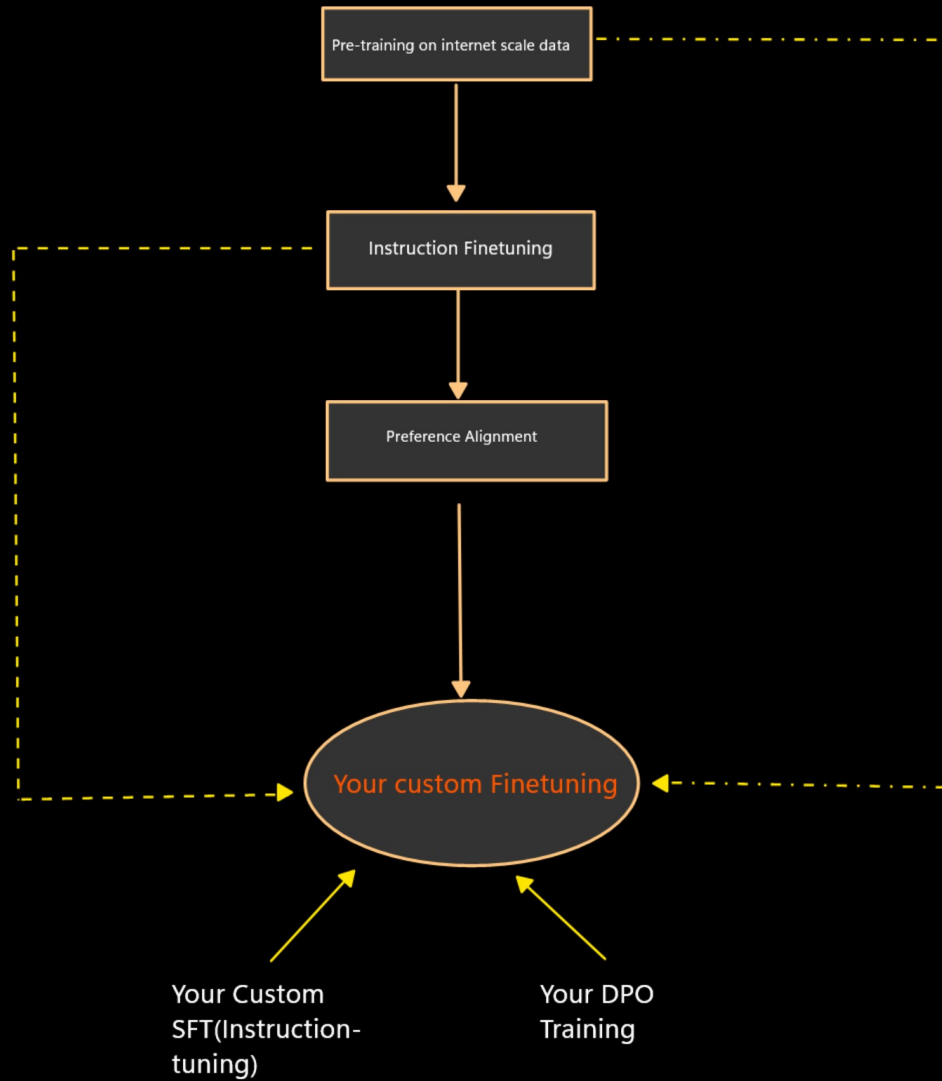


Understand LLM training
Transfer learning and model fine-tuning
Why fine-tuning was difficult in LSTM models
LSTM vs Transformer
Fine-tuning BERT
LLM knowledge distillation
LLM quantization
LLM fine-tuning with non-instructional data (raw PDF, TXT, documents)
Instruction fine-tuning
Preference alignment
Fine-tuning using tools: Hugging Face, LLaMA Factory, Unsloth, Axolotl
GPT fine-tuning
Gemini fine-tuning
SLM fine-tuning
Multimodal fine-tuning
Embedding fine-tuning

1. Training stages of LLMs and where this LoRA exist?
2. Full Parameter Fine-Tuning vs PEFT (Parameter Efficient Fine-Tuning) and methods to achieve PEFT
3. What are Weights in Neural Networks and Transformer Architecture?
4. Matrix and Rank in Matrix
5. What is LoRA or LoRA Adapter and benefits of using LoRA
6. What is Optimizer (Weight Update Concept)?
7. Practical of LORA

Understand LLM training



meta-llama/Llama-3.2-1B

meta-llama/Llama-3.2-1B-Instruct

alignment-handbook/zephyr-7b-dpo-full

Full-parameter Fine-tuning

Updates all the model weights

The weight matrices are very large

7B model has 7 billion weights, 13B has 13 billion, and so on

All weights get updated repeatedly for multiple "epochs"

Storing and updating weights requires a lot of memory
which limits fine-tuning to large GPUs or GPU clusters

Expensive Training

Parameter-Efficient Fine-tuning (PEFT)

Does not update all the model weights

Original weight matrices are frozen

Only a small number of additional weights(parameters) are trained

Way of achieve PEFT:

1. LoRA (Low-Rank Adaptation) → QLoRA (Quantized Low-Rank Adaptation)
2. DoRA (Weight-Decomposed Low-Rank Adaptation)
3. Prefix Tuning (adds learnable prefix vectors)
4. Prompt Tuning (learns input embeddings)
5. BitFit (trains only bias parameters)
6. IA³ (scales attention mechanisms)

Example: LoRA trains small low-rank matrices instead of full weights Training updates only these few parameters repeatedly for multiple epochs

LoRA = Less memory + Faster training + Lower cost + Easy customization

enabling fine-tuning on single GPUs or even consumer hardware

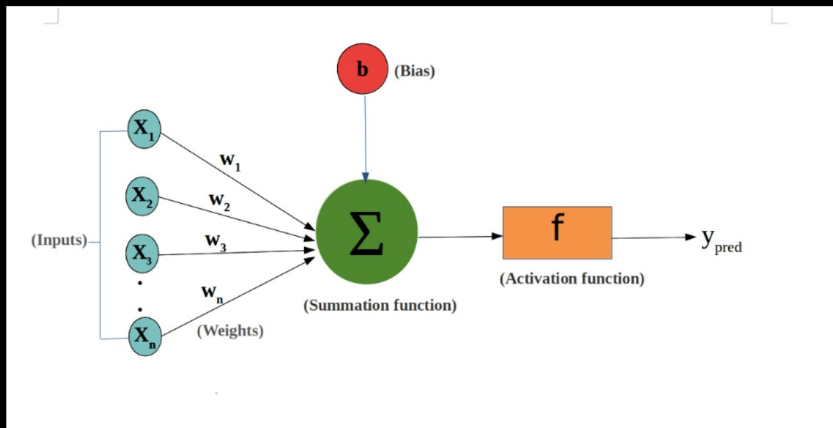
Performance almost same

What is weights ?

Weight = importance score

Neural Net: "Which feature is important?"

Taste = Patty $\times$ 0.9 + Cheese $\times$ 0.7 + Sauce $\times$ 0.5 + Bun $\times$ 0.2



What happens during training?

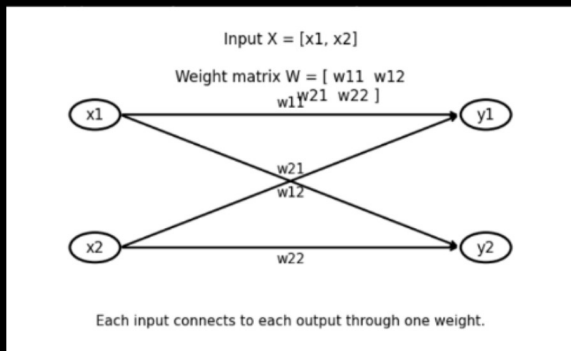
Initially weights are random

During training:

- Loss is calculated
- Weights are updated using gradients

Goal:

Find the best weights



Example:

Input:

$x_1 = 2, x_2 = 3$

Weights:

$w_1 = 0.5, w_2 = 1.0$

Meaning:

Model is deciding:

- Is x_1 important? → lower weight
- Is x_2 important? → higher weight

Weight = "how much to trust this input"

Weights in Transformer Architecture

Transformer: "Which word is important for another word?"

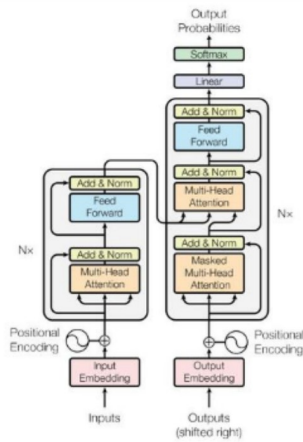
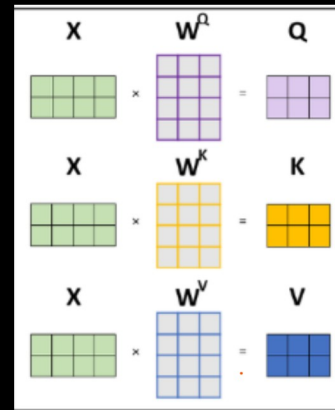


Figure 1: The Transformer - model architecture.



Linear Layers

- WQ (Query)
- WK (Key)
- WV (Value)

These are all weights matrices

Attention Weights (MOST IMPORTANT)

Model decides:

Which word is important for another word

"I love machine learning"

"learning" focuses on which word?

- love? ✓
- machine? ✓✓

This importance = attention weights

Deep Learning = A Matrix Game

Output = Input $\times$ Weights + Bias

Input $\rightarrow$ vector

Weights(Model parameters) $\rightarrow$ matrix

Output $\rightarrow$ transformed vector

Images $\rightarrow$ matrices (pixels)

Videos $\rightarrow$ 3D matrices

Text Embeddings $\rightarrow$ Words $\rightarrow$ vectors

Vocabulary $\rightarrow$ embedding matrix

Attention (Transformers)

Q, K, V $\rightarrow$ matrices

Attention = matrix multiplication

Matrix and Rank in Matrix

Differences between scalar, vector and matrix

Scalar 5

Vector [1,2,3]

Matrix [[1,2]
 [3,4]]

Matrix = vectors ka collection

Vector = special case of matrix (1 row or 1 column)

Matrix = a table of numbers (grid)

A = [1 2
 3 4]

This is called a 2×2 matrix.

2 rows

2 columns

More Examples

1. 3×2 Matrix

$$B = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

This is called a 3×2 matrix.

3 rows

2 columns

2. 2×3 Matrix

$$C = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

This is called a 2×3 matrix.

2 rows

3 columns

3. $m \times n$ Matrix (General Form)

$$D = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ \dots & & & & \\ a_{m1} & a_{m2} & a_{m3} & \dots & a_{mn} \end{bmatrix}$$

This is called an $m \times n$ matrix.

m rows

n columns

Row Matrix

$$R = \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}$$

This is called a 1×4 matrix.

1 row

4 columns

Only one row $\rightarrow$ so it is a row matrix

Column Matrix

$$C = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

This is called a 4×1 matrix.

4 rows

1 column

Only one column $\rightarrow$ so it is a column matrix

What is Rank?

In simple language:

Rank = how many independent rows (or columns) are there

Number of linearly independent rows or columns

How many linearly independent row or column are there?

Meaning:

- Can one row be formed from another row?
- If yes → information is repeating

Real World Analogy

Think:

1000 people

Full rank:

All have different personalities

Low rank:

Only 2–3 types of people, just scaled differently

Visual understanding

Imagine 2D space

Row1 = direction 1

Row2 = direction 2

If both go in different directions → rank = 2

If both go in same direction → rank = 1

General rule

$2 \times 2 \rightarrow \text{max rank } 2$

$3 \times 3 \rightarrow \text{max rank } 3$

$1000 \times 1000 \rightarrow \text{max rank } 1000$

For $m \times n$ matrix:

Maximum rank = $\min(m, n)$

Example 1: Full Rank Matrix

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Let's check:

Is second row a multiple of first row?

First row: (1,2)

If we multiply by 3:

(3,6)

But second row is (3,4)

It does not match

Rows are independent

Rank = 2

Full rank

Example 2: Low Rank Matrix

$$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

Second row = 2 × first row

$$(1,2) \times 2 = (2,4)$$

Second row is not independent

Rank = 1

This is a low rank matrix

Understand from column view

Matrix:

$$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

Columns:

Column1 = (1,2)

Column2 = (2,4)

Column2 = 2 × Column1

Columns are also dependent

So rank = 1

Example 3: 3×3 example

$$C = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix}$$

Notice:

$$\text{Row2} = 2 \times \text{Row1}$$

$$\text{Row3} = 3 \times \text{Row1}$$

All rows are in the same direction

$$\text{Rank} = 1$$

Example 4: 3×3 example

Matrix:

$$D = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Reason:

Rows are not fully independent. There is a linear relation:

$$\text{Row3} = 2 \times \text{Row2} - \text{Row1}$$

because

$$2(4,5,6) - (1,2,3) = (8,10,12) - (1,2,3) = (7,8,9)$$

This means the 3rd row is dependent.

But:

Row1 and Row2 are independent

2 linearly independent row

Therefore total independent rows = 2

Final Answer: Rank = 2

Now the question is what to see to check the rank of the matrix? Row or column?

you can check both — the result will be the same

Concept: Rank = number of linearly independent rows = number of linearly independent columns

$$\begin{bmatrix} \checkmark & \checkmark & \times & \times & \times \\ 1 & 2 & 5 & 3 & 4 \\ 3 & 3 & 9 & 6 & 9 \\ 2 & 3 & 8 & 5 & 7 \\ 4 & 1 & 6 & 5 & 9 \end{bmatrix}$$

$$C_3 = C_1 + 2C_2$$

$$C_4 = C_1 + C_2$$

$$C_5 = 2C_1 + C_2$$

What is a Rank of this matrix?

how many linear independent - row or column are there?

$$\begin{matrix} C_1 & C_2 & C_3 & C_4 & C_5 \\ \begin{bmatrix} 1 & 2 & 5 & 3 & 4 \\ 3 & 3 & 9 & 6 & 9 \\ 2 & 3 & 8 & 5 & 7 \\ 4 & 1 & 6 & 5 & 9 \end{bmatrix} \end{matrix}$$

=

$$\begin{matrix} C_1 & C_2 \\ \begin{bmatrix} 1 & 2 \\ 3 & 3 \\ 2 & 3 \\ 4 & 1 \end{bmatrix} \end{matrix}$$

Store Less Parameter

$$\begin{bmatrix} 1 & 0 & 1 & 1 & 2 \\ 0 & 1 & 2 & 1 & 1 \end{bmatrix}$$

$$4 \times 5 = 20 \text{ Parameters}$$

$$4 \times \underline{2} = 8$$

$$\underline{2} \times \underline{5} = 10$$

Rank

$$8 + 10 = 18 \text{ Parameters}$$

Low Rank $\longrightarrow$ which is used in LoRA

$$W$$

$$d \times k$$

=

$$A$$

$$d \times \underline{r}$$

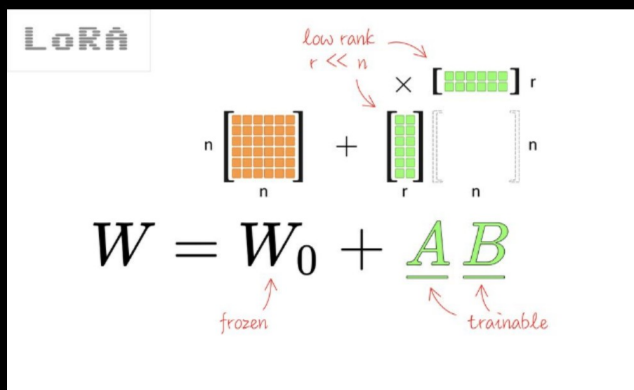
.

$$B$$

$$\underline{r} \times k$$

Rank

$$r < \min[d, k]$$



LoRA trains small low-rank matrices instead of full weights

These weight changes are tracked in two separate, smaller matrices that get multiplied together to form a matrix the same size as the model's weight matrix.

LoRA says:

"It is possible that the weight update matrix is not full rank."

Its rank can be small.

LoRA: Learn two small matrices B and A.

And finally:

$$W_{\text{new}} = W_0 + BA$$

Low-rank update = updating a large matrix by approximating it using smaller matrices.

LoRA

QLoRA(Quantized LoRA)

- This is LoRA 2.0
- Uses even less memory with "recoverable" quantization

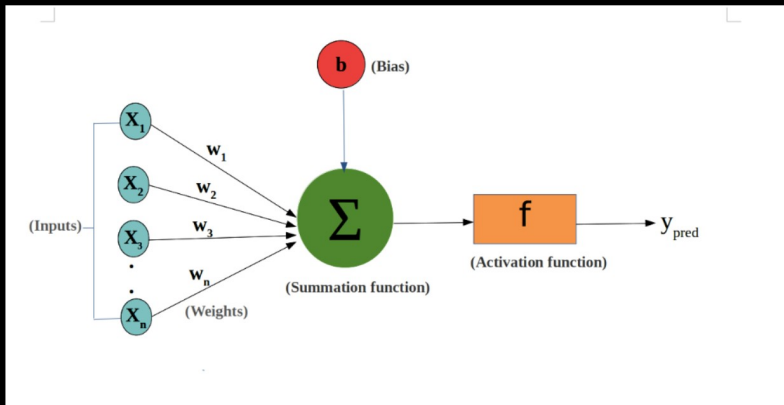
Why we should use LoRA?

GPU memory requirements are drastically reduced

Training becomes much faster (even on a single GPU)

Fine-tuning becomes affordable (ideal for startups and individuals)

You can use different LoRA adapters for multiple tasks without duplicating the full model



BP $\longrightarrow$ Back Propagation

Update weights using optimizers

GD / SGD: Simple update
 GD / SGD with Momentum: Smooth direction
 RMSProp: Adaptive learning
 Adam: Smart + fastest convergence
 AdamW(Weight Decay) + Learning Rate Scheduler + Warmup

GD (Gradient descent)

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W} \longrightarrow L = (\gamma - \hat{\gamma})^2 \longrightarrow \hat{\gamma} = \sigma \left(\sum_{i=1}^n w_i x_i \right)$$

$$\downarrow$$

$$L = \left[\gamma - \sigma \left(\sum_{i=1}^n w_i x_i \right) \right]^2$$

LoRA $\longrightarrow W = W_0 + \boxed{\Delta W}$

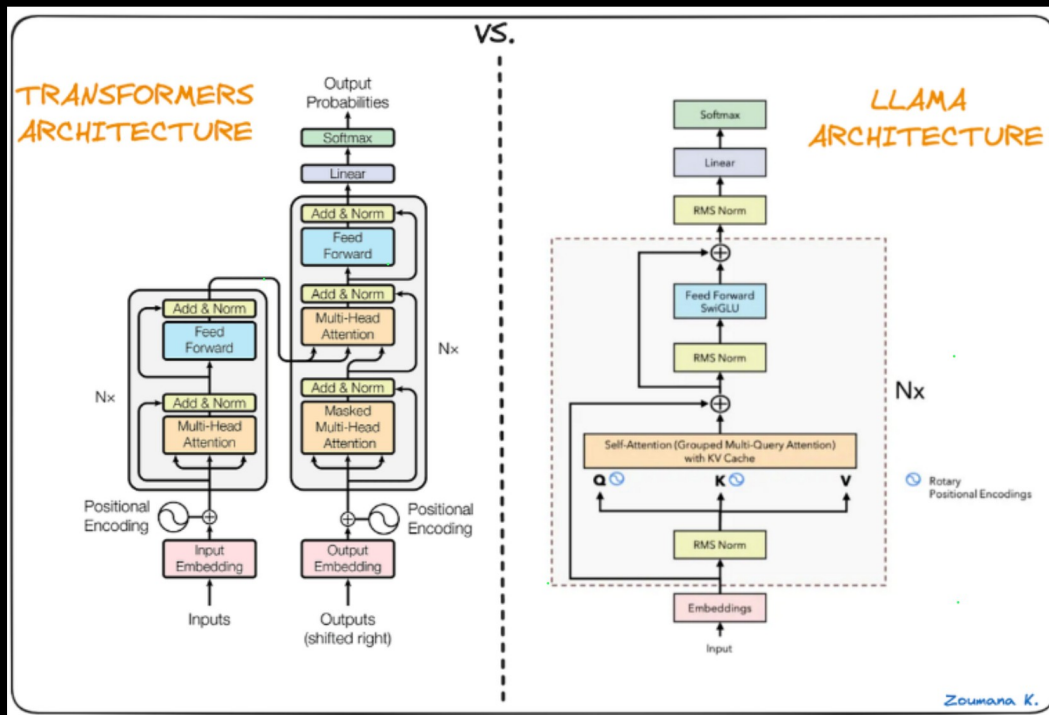
$W = W_0 + BA$

this Parameter (weight or adapter) will be updated

this is adapter
 or this is weight
 (Parameter)

How it will be updated using : AdamW
optimizer

Where is LoRA applied in LLMs(Where this adapter is going to be attached)?



What is a Transformer?

Attention + Neural Network

"A neural network architecture built around self-attention"
there:

- Loss is computed
- Gradient is calculated

In Transformer attention, matrices:

- W_q , W_k , W_v , W_o

Paper mostly applies LoRA on:

- W_q and W_v (query & value)

Best performance under same parameter budget:

- W_q alone → average
- W_v alone → average
- But $W_q + W_v$ → best overall

So practical rule:

$q_proj + v_proj$ (most common industry default today too)

